

Universidad Galileo	Guatemala 21 de Septiembre del 2026
Facultad: FISICC	Alumno: Roberto Molina
Curso: Microprocesadores	Carnet: 24000849
Sección: A	Hora de Laboratorio: 1:00 a 1:50
Auxiliar: Brandon Mendoza	Día de Laboratorio: Lunes

Laboratorio #6 Puertos Paralelos (GPIO)

Resumen:

En la práctica de laboratorio se trabajó con los puertos paralelos GPIO, utilizando programación en lenguaje C. El objetivo principal fue aprender a configurar y controlar pines digitales como entradas y salidas, empleando los registros internos del microcontrolador.

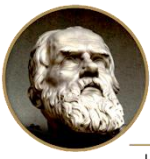
Se configuraron como entradas los pines conectados a cinco pulsadores, equivalentes a las cinco direcciones del joystick, y como salidas los pines conectados a los LEDs. El funcionamiento consistió en detectar cuándo se presionaba cada pulsador y encender el LED correspondiente, manteniéndolo apagado mientras el pulsador se encontraba en reposo.

Para realizar la configuración fue necesario habilitar previamente la señal de reloj de los módulos GPIO mediante los registros del sistema RCC, además de utilizar registros como MODER para definir el modo de los pines, PUPDR para configurar resistencias internas de pull-up o pull-down, IDR para leer las entradas y ODR para controlar las salidas.

Finalmente, se implementó un ciclo continuo que permitió leer el estado de los pulsadores y controlar los LEDs en tiempo real, aplicando operaciones a nivel de bits y directivas del preprocesador para facilitar el manejo de los registros GPIO.

Objetivos:

- Configurar correctamente los puertos GPIO del microcontrolador STM32, definiendo los pines necesarios como entradas y salidas mediante sus registros de configuración.



- Leer el estado de los pulsadores y controlar los LEDs correspondientes, de manera que cada pulsador active el LED asignado.
- Aplicar operaciones a nivel de bits y directivas de preprocesador en lenguaje C para manipular de forma eficiente los registros IDR, ODR, MODER y PUPDR del microcontrolador.

Teoría:

Tarjeta NUCLEO-G431KB

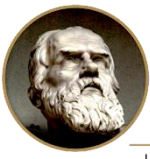
La NUCLEO-G431KB es una tarjeta de desarrollo de STMicroelectronics que usa el microcontrolador STM32G431KBT6, con núcleo Arm Cortex-M4 a 170 MHz en un encapsulado UFQFPN32, con 128 KB de memoria Flash y 32 KB de SRAM [1]. No necesita un programador externo, ya que trae integrado el depurador/programador STLINK-V3E [1]. La serie STM32G4 es una familia de microcontroladores con distintos tamaños de memoria, encapsulados y periféricos [2], por lo que la cantidad de puertos disponibles depende del modelo. Al tener solo 32 pines, esta tarjeta no expone todos los puertos, así que los pines de los pulsadores y de los LEDs se deben elegir entre los que sí están disponibles [1].

Puertos paralelos de propósito general (GPIO)

Los microcontroladores STM32G4 cuentan con hasta siete puertos paralelos bidireccionales de 16 pines cada uno, aunque esto depende de la versión del chip [2]. Cada pin se puede configurar por software como entrada digital, salida digital, función alternativa o entrada/salida analógica, escribiendo en los registros del módulo GPIO [2].

Registros principales del GPIO

- **GPIOx_MODER (Mode Register):** define el modo de cada pin usando 2 bits por pin: 00 = entrada (estado después del reset), 01 = salida de propósito general, 10 = función alternativa y 11 = analógico. Para el pin n se modifican los bits $2n$ y $2n+1$; por ejemplo, el pin 10 usa los bits 20 y 21 [2].
- **GPIOx_PUPDR (Pull-up/Pull-down Register):** usa 2 bits por pin: 00 = sin resistencia, 01 = pull-up, 10 = pull-down y 11 = reservado [2].
- **GPIOx_IDR (Input Data Register):** contiene el estado lógico actual de los pines configurados como entrada [2].



- **GPIOx_ODR (Output Data Register):** lo que se escribe aquí aparece en los pines configurados como salida [2].
- **GPIOx_BSRR (Bit Set/Reset Register):** permite hacer set y reset de cada pin de salida de forma individual. Los bits 0 a 15 ponen en 1 el bit correspondiente del ODR y los bits 16 a 31 lo ponen en 0. Escribir un 0 no tiene efecto, y si se activan el set y el reset del mismo pin, el set tiene prioridad [2]. Esto permite cambiar un pin sin tener que leer primero el registro.
- **GPIOx_AFRL / GPIOx_AFRH (Alternate Function Registers):** asignan la función alternativa de cada pin con 4 bits por pin [2]. En esta práctica no se utilizan, porque los pines trabajan como puerto paralelo.

Resistencias pull-up y pull-down

Cuando un pin está configurado como entrada y no está conectado a nada, su valor queda indefinido y puede cambiar por ruido. Para evitarlo se usan resistencias pull-up o pull-down, que se colocan en las entradas digitales "para fijar un valor por defecto" cuando no se detecta ninguna señal [3]. La pull-up deja la entrada en nivel alto ("1") cuando el circuito está en reposo, y la pull-down la deja en nivel bajo ("0") [3].

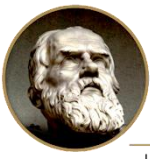
Los STM32 tienen estas resistencias integradas y se activan por medio del registro PUPDR, siempre que el pin esté configurado como entrada digital [2]. Con pull-down, el pin lee '0' en reposo y '1' cuando se presiona el pulsador; con pull-up ocurre lo contrario.

Arquitectura de buses AMBA y señal de reloj

La arquitectura AMBA de Arm define varios buses, entre ellos el AHB y el APB. El AHB es la columna vertebral del sistema y está pensado para componentes de alto rendimiento y alta frecuencia, mientras que el APB es una interfaz simplificada para periféricos que no necesitan tanto ancho de banda [4]. Ambos buses se comunican por medio de un puente, y los periféricos conectados al APB funcionan como esclavos [4]. En el STM32G4 el módulo GPIO está conectado al bus AHB2, por lo que antes de usar un puerto se debe habilitar su reloj en el registro RCC->AHB2ENR [2].

CMSIS

CMSIS (Common Microcontroller Software Interface Standard) es un conjunto de APIs, componentes de software, herramientas y flujos de trabajo que facilitan la reutilización de código y reducen la curva de aprendizaje de quienes programan microcontroladores [5].



No es una capa pesada de software que agregue sobrecarga, ni define periféricos estándar [5]. Gracias a CMSIS, los archivos de cabecera del fabricante definen los símbolos y estructuras de cada periférico, lo que permite escribir directamente expresiones como GPIOA->ODR o RCC->AHB2ENR en C.

Operaciones a nivel de bits y máscaras

Para modificar solo ciertos bits de un registro se usan operadores a nivel de bits. El AND (&) da 1 solo si ambos bits son 1; el OR (|) da 1 si al menos uno de los bits es 1. Con una máscara como $1 \ll 2$ se puede activar un bit con $|=$ y desactivarlo con $\&= \sim$ [6]. De esta forma:

- registro $|= (1 \ll n)$; pone en 1 el bit n sin tocar los demás.
- registro $\&= \sim(1 \ll n)$; pone en 0 el bit n sin tocar los demás.
- if (GPIOx->IDR & (1<<n)) es verdadero solo si el pin n está en alto.

Macros en C

La directiva #define crea una macro, que es "la asociación de un identificador o identificador parametrizado con una cadena de token" [7]. Cuando la macro lleva parámetros entre paréntesis, se le llama macro de tipo función [7].

LEDs y resistencia limitadora de corriente

Un LED siempre debe tener una resistencia en serie, ya que sin ella el LED "intenta extraer la máxima corriente posible" de la fuente [8]. El valor se obtiene con la ley de Ohm: se resta el voltaje directo del LED al voltaje de alimentación y el resultado se divide entre la corriente deseada [8]:

$$R = (V_{\text{fuente}} - V_f) / I_{\text{LED}}$$

Resultados prácticos:

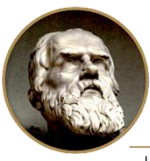
```
1  #include "RTE_Components.h"
2  #include CMSIS_device_header
3
4  //macro para generar la mascara de un bit
5  #define MASK(x) (1 << (x))
6
7  /* Pines de los pulsadores entradas (todos en CN4) */
8  #define Pin_Boton1  0  /* PB0 */
9  #define Pin_Boton2  4  /* PB4 */
10 #define Pin_Boton3  5  /* PB5 */
11 #define Pin_Boton4  6  /* PB6 */
12 #define Pin_Boton5  8  /* PA8 */
13
14 /* Pines de los LEDs salidas (todos en CN3) */
15 #define Pin_LED1    1  /* PA1 */
16 #define Pin_LED2    0  /* PA0 */
17 #define Pin_LED3    4  /* PA4 */
18 #define Pin_LED4    5  /* PA5 */
19 #define Pin_LED5    6  /* PA6 */
20
```

Fig. 1. Definición de macro y declaración de constantes

```
int main() {

    // habilitar el reloj de los puertos A y B
    RCC -> AHB2ENR |= RCC_AHB2ENR_GPIOAEN ;
    RCC -> AHB2ENR |= RCC_AHB2ENR_GPIOBEN;
```

Fig. 2. Habilitar relojes



```
//botones como entrada 00
//& para apagar los bits de la posicion del pin y dejarlo en 00
GPIOB -> MODER &= ~(MASK(2*Pin_Boton1) | MASK(2*Pin_Boton1+1));
GPIOB -> MODER &= ~(MASK(2*Pin_Boton2) | MASK(2*Pin_Boton2+1));
GPIOB -> MODER &= ~(MASK(2*Pin_Boton3) | MASK(2*Pin_Boton3+1));
GPIOB -> MODER &= ~(MASK(2*Pin_Boton4) | MASK(2*Pin_Boton4+1));
GPIOA -> MODER &= ~(MASK(2*Pin_Boton5) | MASK(2*Pin_Boton5+1));

//leds como salida 01
//& para limpiar ambos bits de la posicion del pin y dejarlos en 00
//| para poner en 01 el bit de la posicion del pin
GPIOA -> MODER &= ~(MASK(2*Pin_LED1) | MASK(2*Pin_LED1+1));
GPIOA -> MODER |= MASK(2*Pin_LED1);

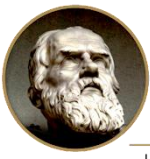
GPIOA -> MODER &= ~(MASK(2*Pin_LED2) | MASK(2*Pin_LED2+1));
GPIOA -> MODER |= MASK(2*Pin_LED2);

GPIOA -> MODER &= ~(MASK(2*Pin_LED3) | MASK(2*Pin_LED3+1));
GPIOA -> MODER |= MASK(2*Pin_LED3);

GPIOA -> MODER &= ~(MASK(2*Pin_LED4) | MASK(2*Pin_LED4+1));
GPIOA -> MODER |= MASK(2*Pin_LED4);

GPIOA -> MODER &= ~(MASK(2*Pin_LED5) | MASK(2*Pin_LED5+1));
GPIOA -> MODER |= MASK(2*Pin_LED5);
```

Fig. 3. Configurar entradas y salidas



```
//habilitar pull-down 10
//& para limpiar ambos bits de la posicion del pin y dejarlos en 00
//| para poner en 10 el bit de la posicion del pin
//lo de pull down solo tiene sentido en registros de entrada, por eso se hace solo para los botones
GPIOB -> PUPDR &= ~(MASK(2*Pin_Boton1) | MASK(2*Pin_Boton1+1));
GPIOB -> PUPDR |= MASK(2*Pin_Boton1+1);

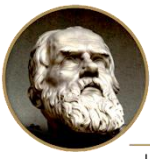
GPIOB -> PUPDR &= ~(MASK(2*Pin_Boton2) | MASK(2*Pin_Boton2+1));
GPIOB -> PUPDR |= MASK(2*Pin_Boton2+1);

GPIOB -> PUPDR &= ~(MASK(2*Pin_Boton3) | MASK(2*Pin_Boton3+1));
GPIOB -> PUPDR |= MASK(2*Pin_Boton3+1);

GPIOB -> PUPDR &= ~(MASK(2*Pin_Boton4) | MASK(2*Pin_Boton4+1));
GPIOB -> PUPDR |= MASK(2*Pin_Boton4+1);

GPIOA -> PUPDR &= ~(MASK(2*Pin_Boton5) | MASK(2*Pin_Boton5+1));
GPIOA -> PUPDR |= MASK(2*Pin_Boton5+1);
```

Fig. 4. Habilitar resistencias Pull-Down



```
while (1){  
  
    // si el boton esta presionado, se enciende el led correspondiente  
    // IDR input data register, ODR output data register  
    // lee IDR y si el bit correspondiente al pin del boton es 1, enciende el led correspondiente, sino lo apaga  
    if(GPIOB->IDR & MASK(Pin_Boton1)){  
        GPIOA->ODR |= MASK(Pin_LED1);  
    } else {  
        GPIOA->ODR &= ~MASK(Pin_LED1);  
    }  
  
    //boton 2  
    if(GPIOB->IDR & MASK(Pin_Boton2)){  
        GPIOA->ODR |= MASK(Pin_LED2);  
    }else {  
        GPIOA->ODR &= ~MASK(Pin_LED2);  
    }  
  
    //boton 3  
    if(GPIOB->IDR & MASK(Pin_Boton3)){  
        GPIOA->ODR |= MASK(Pin_LED3);  
    }else {  
        GPIOA->ODR &= ~MASK(Pin_LED3);  
    }  
  
    //boton 4  
    if(GPIOB->IDR & MASK(Pin_Boton4)){  
        GPIOA->ODR |= MASK(Pin_LED4);  
    }else {  
        GPIOA->ODR &= ~MASK(Pin_LED4);  
    }  
  
    //boton 5  
    if(GPIOA->IDR & MASK(Pin_Boton5)){  
        GPIOA->ODR |= MASK(Pin_LED5);  
    }else {  
        GPIOA->ODR &= ~MASK(Pin_LED5);  
    }  
  
}
```

Fig. 5. Comportamiento de los LEDs

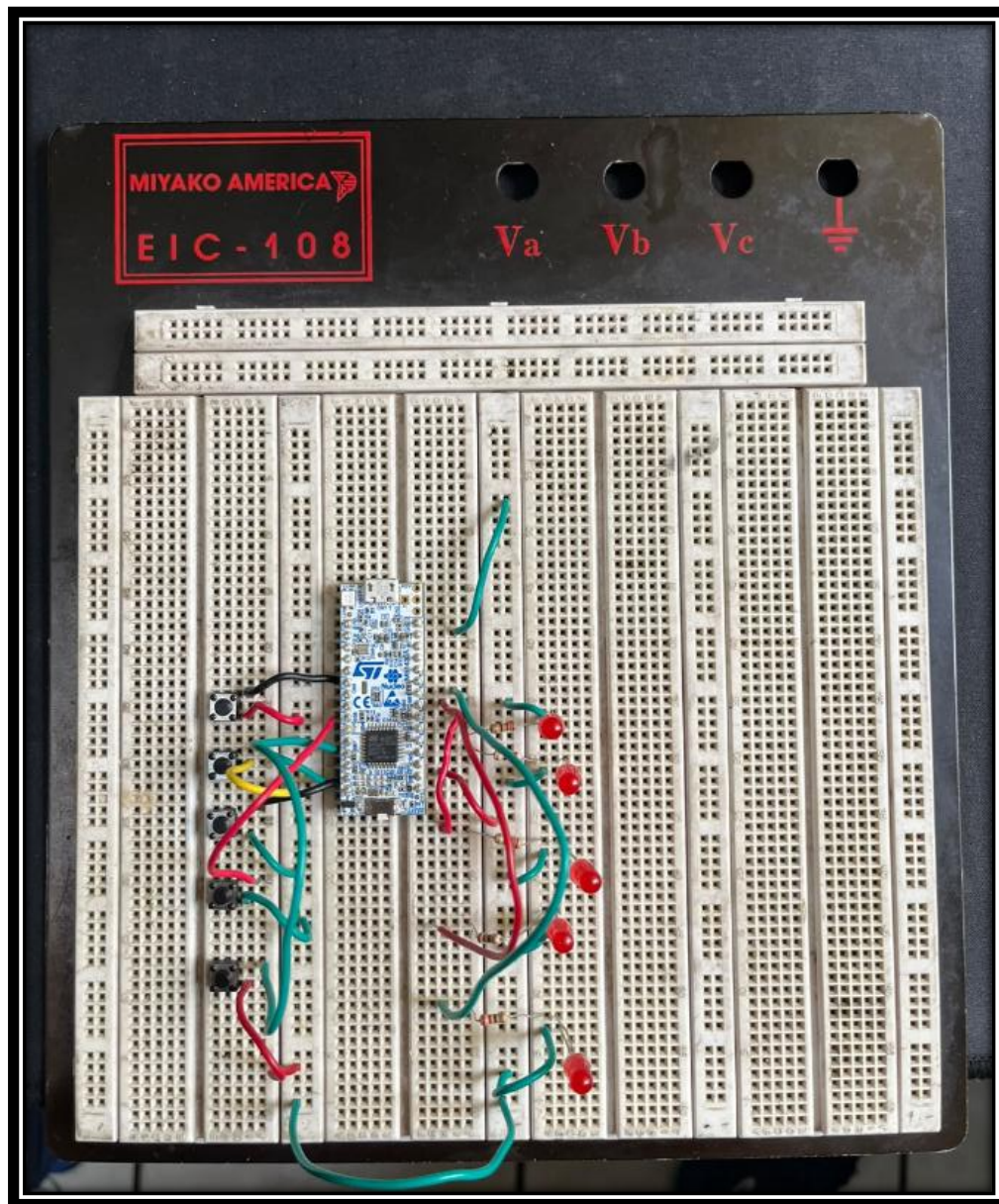
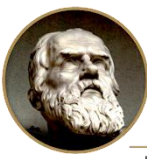


Fig. 6. Circuito

Video demostrativo: <https://youtu.be/1Q6eos66YWQ>

Procedimientos teóricos:

Definición de la macro MASK y de los pines (Fig. 1)

Después de incluir RTE_Components.h y el header del dispositivo CMSIS, se definió la macro MASK(x) como $(1 \ll (x))$.

¿Qué hace la macro?

Genera un número con un solo bit en 1, en la posición x, y todos los demás en 0. Por ejemplo, MASK(4) equivale a 0x10. Ese número funciona como máscara para encender, apagar o leer un bit específico de un registro sin afectar a los demás.

¿Qué tipo de función es?

Es una macro de tipo función del preprocesador de C. No es una función real: no se llama en tiempo de ejecución, no ocupa la pila y no revisa tipos de datos. Antes de compilar, el preprocesador reemplaza cada MASK(...) por el texto $(1 \ll (...))$.

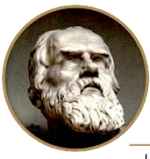
¿Qué diferencia hay con el lado derecho de la asignación?

En cuanto al resultado, ninguna. Después del preprocesado, GPIOA->ODR |= MASK(Pin_LED1); queda como GPIOA->ODR |= $(1 \ll (1))$; y el compilador genera exactamente el mismo código. La ventaja de la macro es que el código se lee mejor, porque deja claro que se está creando una máscara, y evita repetir la misma expresión muchas veces. Como la macro solo sustituye texto, a diferencia de la versión de la guía $(1 \ll x)$, el parámetro se dejó entre paréntesis, (x), para que expresiones como MASK(2*Pin_Boton1+1) se evalúen completas antes del desplazamiento.

También se usaron directivas #define para darle nombre a cada pin. Los pulsadores se conectaron en PB0, PB4, PB5, PB6 y PA8 (conector CN4), y los LEDs en PA1, PA0, PA4, PA5 y PA6 (conector CN3). Así, si se cambia un pin en el circuito, solo hay que modificar una línea.

Habilitación de relojes (Fig. 2)

Al inicio de main() se habilitó el reloj de los puertos A y B, ya que en el STM32G431 el módulo GPIO está conectado al bus AHB2. Para esto se escribió sobre el registro RCC->AHB2ENR con los símbolos RCC_AHB2ENR_GPIOAEN y RCC_AHB2ENR_GPIOBEN definidos por CMSIS. Se usó una operación OR (|=) para encender solo esos bits, sin habilitar ni deshabilitar por error otros módulos.



Configuración de entradas y salidas (Fig. 3)

Cada pin usa 2 bits en el registro MODER, por lo que el pin n ocupa los bits $2n$ y $2n+1$. Para los pulsadores, esos dos bits se dejaron en 00 (entrada) con una operación AND con el complemento de ambas máscaras: $\&= \sim(\text{MASK}(2*n) | \text{MASK}(2*n+1))$.

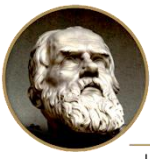
Para los LEDs se necesitaba el valor 01 (salida). Primero se limpiaron ambos bits de la misma forma y luego se puso en 1 solo el bit $2n$ con $|= \text{MASK}(2*n)$. Limpiar antes de escribir es importante en el puerto A, porque después del reset varios de sus pines no quedan en 00. Si solo se hiciera el OR, algunos pines podrían quedar en 11 (modo analógico) en lugar de salida.

Resistencias pull-down (Fig. 4)

En el registro PUPDR se configuró la resistencia interna de pull-down de cada pulsador, que corresponde al valor 10. Se siguió el mismo procedimiento: primero se limpiaron los dos bits del pin y luego se puso en 1 el bit $2n+1$ con $|= \text{MASK}(2*n+1)$. Esta configuración solo se aplicó a los pulsadores, porque las resistencias pull-up y pull-down solo tienen sentido en pines de entrada. Con pull-down, el pin lee 0 cuando el pulsador está en reposo y 1 cuando se presiona.

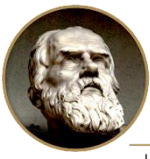
Comportamiento de los LEDs (Fig. 5)

Dentro del ciclo while(1) se revisa cada pulsador con su propio if. Para cada uno se lee el registro IDR del puerto correspondiente y se aplica un AND con la máscara del pin, por ejemplo $\text{GPIOB} \rightarrow \text{IDR} \& \text{MASK}(\text{Pin_Boton1})$. Si el resultado es distinto de cero, el pulsador está presionado y el LED se enciende con $\text{GPIOA} \rightarrow \text{ODR} |= \text{MASK}(\text{Pin_LED1})$. Si no, el LED se apaga con $\text{GPIOA} \rightarrow \text{ODR} \&= \sim \text{MASK}(\text{Pin_LED1})$. El mismo procedimiento se repite para los cinco pulsadores; en el caso del botón 5, el IDR que se lee es el del puerto A, porque ese pulsador está en PA8.



Conclusiones:

- El armado del circuito resultó un poco complicado, ya que se sentía nerviosismo al momento de conectar los componentes por temor a dañar el microcontrolador. Además, fue necesario buscar una imagen del pinout de la tarjeta para identificar cuál era cada pin, porque los nombres que aparecen en la placa no coinciden directamente con los del puerto (por ejemplo, PA1 o PB4).
- Al principio costó entender que cada pin usa dos bits en los registros MODER y PUPDR, y que no basta con escribir un 1 en la posición del pin. Fue necesario limpiar primero ambos bits y luego escribir el valor deseado, lo que ayudó a comprender mejor por qué las operaciones a nivel de bits son tan importantes al trabajar directamente con los registros.
- El uso de directivas `#define` para nombrar los pines y de la macro MASK hizo que el código fuera mucho más fácil de leer y de corregir. Cuando hubo que cambiar algún pin en el circuito, solo fue necesario modificar una línea, en lugar de buscar y cambiar cada número dentro del programa.



Bibliografía:

- [1] STMicroelectronics, "STM32G4 Nucleo-32 board (MB1430)", Manual de usuario UM2397, Rev. 2, sep. 2019. Accedido el 3 de octubre de 2026. [En línea]. Disponible: https://www.st.com/resource/en/user_manual/um2397-stm32g4-nucleo32-board-mb1430-stmicroelectronics.pdf
- [2] STMicroelectronics, "STM32G4 Series advanced Arm-based 32-bit MCUs", Manual de referencia RM0440. Accedido el 3 de octubre de 2026. [En línea]. Disponible: https://www.st.com/resource/en/reference_manual/rm0440-stm32g4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
- [3] "2.2.3 Pull-up y pull-down · GitBook". CATEDU. Accedido el 3 de octubre de 2026. [En línea]. Disponible: https://catedu.github.io/programa-arduino-mediante-codigo/resistencias_pullup_y_pulldown.html
- [4] "The Advanced Microcontroller Bus Architecture: An Introduction". All About Circuits. Accedido el 3 de octubre de 2026. [En línea]. Disponible: <https://www.allaboutcircuits.com/technical-articles/introduction-to-the-advanced-microcontroller-bus-architecture/>
- [5] Arm Ltd., "CMSIS – Common Microcontroller Software Interface Standard, Version 6". Arm Software. Accedido el 3 de octubre de 2026. [En línea]. Disponible: https://arm-software.github.io/CMSIS_6/latest/General/index.html
- [6] L. Llamas, "Operadores bitwise en C++ para manipular bits". Luis Llamas. Accedido el 3 de octubre de 2026. [En línea]. Disponible: <https://www.luisllamas.es/cpp-operadores-bitwise/>
- [7] "Directiva #define (C/C++)". Microsoft Learn. Accedido el 3 de octubre de 2026. [En línea]. Disponible: <https://learn.microsoft.com/es-es/cpp/preprocessor/hash-define-directive-c-cpp?view=msvc-170>
- [8] "Resistencias limitadoras de corriente". PCBasic. Accedido el 3 de octubre de 2026. [En línea]. Disponible: https://www.pcbasic.com/es/blog/current_limiting_resistors.html